

FluxDrop Project Architecture Brief

This document is for interview preparation. It explains FluxDrop in plain English, maps the files to the app layers, explains Firebase/session behavior, and lists what can be improved if the project becomes production-grade.

1. One-Line Project Explanation

FluxDrop is a Flutter mobile app that lets one phone send files to another phone using a short human-friendly code.

The receiver does not need an email, phone number, or password. Each app install gets an anonymous Firebase identity and a 6-character short code such as `XFVTH9`.

2. The Main User Flow

1. The app starts.
2. Firebase initializes.
3. The app restores or creates an anonymous user.
4. The user gets a short code.
5. A sender enters the receiver's short code.
6. The app checks Firestore to make sure that code exists.
7. The sender selects one or more files.
8. A transfer document is created in Firestore.
9. Files are uploaded to Firebase Storage.
10. The receiver sees the incoming transfer through a Firestore real-time listener.
11. The receiver accepts or declines.
12. If accepted, files download to the receiver device.
13. The app checks file integrity using SHA-256.
14. The transfer becomes completed.

3. Architecture Style

The app uses a pragmatic layered Flutter architecture.

It is not strict MVVM because there is no dedicated ViewModel layer. Some screen files still hold UI state and orchestration logic. The architecture is better described as:

- Presentation layer: Flutter screens and reusable widgets.
- Model layer: Dart classes that represent users, transfers, and files.
- Service layer: Firebase, transfer, notification, network, and native-channel work.
- Native/platform layer: Kotlin and Swift implementations behind Flutter MethodChannels.
- Backend layer: Firebase Auth, Firestore, Firebase Storage, FCM, and Cloud Functions.

4. Why This Architecture Was Used

This architecture was used because it is practical for an assessment-sized mobile app.

The UI stays in `screens/`, repeated UI controls stay in `widgets/`, data shapes stay in `models/`, and Firebase/native logic stays in `services/`. That makes the project easier to explain and modify without requiring a heavy state-management setup.

The main tradeoff is that `TransferService` became a large coordinator. It handles upload, download, progress, cancellation, fallback, hashing, recovery, and storage checks. That is acceptable for the assessment, but in a production app it should be split into smaller pieces.

5. Layer Map: What Is Present Where

Layer	Files	Responsibility
App entry	<code>lib/main.dart</code>	Starts Flutter, initializes Firebase, applies theme, registers Android FCM background handler, opens <code>SplashScreen</code> .
Startup/session	<code>lib/screens/splash_screen.dart</code>	Restores or creates user identity, recovers stale transfers, initializes notifications, moves into <code>HomeScreen</code> .
Presentation/screens	<code>lib/screens/home_screen.dart</code>	Main app shell, code card, incoming transfer prompt, bottom navigation, drawer, theme sheet.
Presentation/screens	<code>lib/screens/send_screen.dart</code>	Recipient lookup, file picking, validation, upload progress, send cancellation, success UI.
Presentation/screens	<code>lib/screens/transfers_screen.dart</code>	Incoming/outgoing transfer lists, accept, decline, download, cancel download, re-download, open/share/save UI.
Presentation/screens	<code>lib/screens/settings_screen.dart</code>	Save location settings, cache cleanup, theme/settings-related controls.
Presentation/screens	<code>lib/screens/file_preview_screen.dart</code>	Opens supported downloaded files inside the app and allows share/open behavior.
Presentation/screens	<code>lib/screens/about_screen.dart</code>	Human-readable project explanation, architecture notes, limitations, developer profile.
Reusable UI	<code>lib/widgets/flux_ui.dart</code>	Shared UI wrappers such as <code>FluxButton</code> and <code>FluxSurface</code> , including NeoPOP theme behavior.
Theme/core	<code>lib/core/theme.dart</code>	App colors, Material theme, theme persistence, system UI styling.
Constants/core	<code>lib/core/constants.dart</code>	Short-code alphabet, file-size limit, transfer TTL, Firestore collection names, status strings.
Models	<code>lib/models/user_model.dart</code>	Represents one app user: Firebase UID, short code, creation date, FCM token.
Models	<code>lib/models/transfer_model.dart</code>	Represents one transfer and its files, status, progress, TTL, LAN endpoint, error message.
Auth service	<code>lib/services/auth_service.dart</code>	Anonymous sign-in, user document creation, unique short-code generation, short-code lookup, FCM token update.
Transfer service	<code>lib/services/transfer_service.dart</code>	Core transfer engine: send, upload, download, progress, cancel, hash verification, stale recovery, expiry, Firebase/LAN fallback.
Notification service	<code>lib/services/notification_service.dart</code>	Local notification setup, Android FCM listeners, notification-tap routing to Transfers.
Network service	<code>lib/services/network_service.dart</code>	Connectivity/mobile-data check used for metered-connection warnings.
LAN service	<code>lib/services/lan_transfer_service.dart</code>	Nearby same-Wi-Fi TCP server/client path and local IP detection.
Android native	<code>android/app/src/main/kotlin/com/example/fluxdrop/MainActivity.kt</code>	MethodChannel methods: <code>getWifiIp</code> , <code>getFreeSpace</code> , <code>saveToGallery</code> using <code>MediaStore</code> .

Layer	Files	Responsibility
iOS native	<code>ios/Runner/AppDelegate.swift</code>	MethodChannel methods: <code>getFreeSpace</code> , <code>saveToGallery</code> using <code>PHPhotoLibrary</code> .
Backend function	<code>functions/index.js</code>	Firestore trigger that sends FCM notification when transfer status becomes <code>uploaded</code> .

6. Important Terms Used In This Project

Layered architecture: Code is separated by responsibility. UI, models, services, native code, and backend logic are kept in different places.

Model: A class that represents data. For example, `UserModel` represents a user and `TransferModel` represents a file transfer.

Service: A class that performs app work. For example, `AuthService` handles sign-in and `TransferService` handles file transfer behavior.

State machine: A controlled status flow. A transfer moves through statuses like `uploading`, `uploaded`, `downloading`, and `completed`.

Firestore listener: A real-time subscription to Firestore updates. It lets the UI update without manual refresh.

Firebase Storage: Cloud storage for actual file bytes.

FCM: Firebase Cloud Messaging. It sends push notifications.

Cloud Function: Server-side JavaScript code that runs automatically when a Firestore event happens.

MethodChannel: A bridge that lets Dart call Android Kotlin or iOS Swift code.

Scoped storage: Android's modern storage privacy system. Apps should save media through approved APIs such as `MediaStore` instead of writing anywhere on the device.

MediaStore: Android API used to save images/videos into public media collections such as `Pictures` or `Movies`.

PHPhotoLibrary: iOS API used to save images/videos into the `Photos` app.

TTL: Time to live. In `FluxDrop`, a transfer waits for 24 hours before it can expire.

SHA-256: A file fingerprint. The app compares sender and receiver hashes to detect corrupted downloads.

7. Startup and Session Handling

What happens in `main.dart`

`main.dart` is the entry point of the app.

It does these things:

1. Calls `WidgetsFlutterBinding.ensureInitialized()` so Flutter plugin/native calls are allowed before the UI starts.
2. Initializes `AppTheme`.
3. Locks the app to portrait mode.
4. Initializes Firebase using `DefaultFirebaseOptions.currentPlatform`.
5. Registers the Android Firebase background message handler.
6. Runs `FluxDropApp`.

The app starts with `SplashScreen`.

What happens in SplashScreen

SplashScreen is not just visual loading. It is the startup coordinator.

It does these things:

1. Checks `AuthService.currentUser`.
2. If Firebase already has a user, it loads that user's Firestore document.
3. If no Firebase user exists, it signs in anonymously.
4. Runs `TransferService().recoverStaleTransfers(user.uid)`.
5. Initializes notifications.
6. If opened from a notification, it starts on the Transfers tab.
7. Navigates to `HomeScreen`.

Why sessions stay consistent

Firebase anonymous auth stores the anonymous user session locally on the device. If the app is closed and reopened, Firebase can restore the same anonymous user. Since the same Firebase UID is restored, the same Firestore user document is loaded, and the same short code appears.

Why iPhone seemed to keep the same code

On iOS, uninstall behavior can be different because iOS/Firebase/keychain behavior may preserve some authentication-related state across reinstall in some local/debug cases. That means the same anonymous Firebase user can sometimes be restored even after reinstalling the app, so the same short code appears again.

This is not something to promise as a product guarantee. The safer explanation is:

"The app treats Firebase anonymous auth as the source of identity. If Firebase restores the same UID, FluxDrop restores the same short code. If the platform clears that UID, FluxDrop creates a new anonymous user and new short code."

Why Android may get a new code after reinstall

On Android, uninstalling the app usually clears app data. If Firebase anonymous auth local state is removed, Firebase no longer knows the old anonymous UID. On the next launch, the app signs in anonymously again, creates a new UID, and generates a new short code.

This is expected for anonymous auth unless an account recovery mechanism exists.

What SharedPreferences is doing

`AuthService` stores `shortCode` and `uid` in `SharedPreferences`, but the primary session source is still Firebase Auth. `SharedPreferences` is local key-value storage. It is useful for small local values, but it is not a full identity recovery system.

If Firebase Auth loses the anonymous UID after reinstall, `SharedPreferences` will usually be gone too, especially on Android uninstall.

8. Identity and Short-Code Flow

The identity flow lives mainly in `AuthService`.

Anonymous sign-in

`signInAnonymously()` calls Firebase Auth anonymous sign-in. Firebase returns a `uid`.

That UID is the stable internal identity for the app install/session.

User document

After sign-in, `_ensureUserModel(uid)` checks:

- Does `users/{uid}` already exist in Firestore?
- If yes, load it and refresh FCM token.
- If no, generate a new short code and create a Firestore user document.

Unique short-code generation

The short code uses constants from `AppConstants`:

- Length: 6
- Alphabet: `ABCDEFGHJKLMNOPQRSTUVWXYZ23456789`

This avoids confusing characters such as 0, Ø, I, 1, and L.

The app generates a code, checks Firestore to see if anyone already has it, and retries if there is a collision.

Short-code lookup

When sender types a code, `lookupByShortCode(code)`:

1. Trims whitespace.
2. Uppercases the code.
3. Queries Firestore `users` where `shortCode == code`.
4. Returns the matching user or `null`.

That is how invalid recipient code is handled early.

9. Transfer Data Model

The central model is `TransferModel`.

A transfer contains:

- `transferId`: unique ID for this transfer.
- `senderId`: Firebase UID of sender.
- `receiverId`: Firebase UID of receiver.
- `senderCode`: sender's short code.
- `receiverCode`: receiver's short code.
- `files`: list of `FileInfo`.
- `status`: current transfer state.
- `uploadProgress`: value between 0.0 and 1.0.
- `downloadProgress`: value between 0.0 and 1.0.
- `createdAt`: creation timestamp.
- `expiresAt`: TTL expiry timestamp.
- `errorMessage`: user/debug-facing failure reason.
- `totalBytes`: total size of all files.

- `transferredBytes`: uploaded bytes.
- `lanIp` and `lanPort`: optional nearby transfer endpoint.

Each `FileInfo` contains:

- file name
- MIME type
- size
- Firebase download URL
- Firebase Storage path
- SHA-256 hash

10. Transfer State Machine

The important transfer statuses are:

Status	Meaning
pending	Default fallback status if unknown.
uploading	Sender is uploading files.
uploaded	Upload finished; receiver can accept/download.
downloading	Receiver is downloading.
completed	Transfer finished successfully.
failed	Upload/download failed or sender cancelled upload.
expired	Transfer passed its TTL.
rejected	Receiver declined the transfer.

Typical happy path:

`uploading -> uploaded -> downloading -> completed`

Important failure paths:

- Sender cancels upload: `uploading -> failed`
- Receiver declines: `uploaded -> rejected`
- Receiver cancels first-time download: `downloading -> uploaded`
- Receiver cancels re-download of completed transfer: `downloading -> completed`
- Transfer too old: `uploaded -> expired`

11. Sending Files

The send flow starts in `SendScreen` and moves into `TransferService.sendFiles()`.

`SendScreen` responsibilities

`SendScreen` owns user-facing send behavior:

- accepts recipient code input
- validates confusing characters

- looks up recipient using `AuthService`
- blocks self-send
- opens file picker
- checks selected files
- warns for large/metered uploads
- calls `TransferService.sendFile`
- shows upload progress
- supports cancel upload

`TransferService.sendFile()` responsibilities

`sendFiles()` does the heavy work:

1. Validates file size.
2. Rejects zero-byte files.
3. Creates a unique transfer ID.
4. Sets transfer TTL to 24 hours.
5. Starts optional LAN TCP server.
6. Creates Firestore transfer document with status `uploading`.
7. Computes SHA-256 hash for each file.
8. Uploads each file to Firebase Storage.
9. Writes progress back to Firestore.
10. Stores file metadata in Firestore.
11. Marks transfer as `uploaded`.

Why Firebase Storage is used

Firestore is not meant for large binary files. Firebase Storage is built for file bytes. Firestore stores metadata and progress, while Storage stores the actual files.

12. Receiving Files

Receiving is mainly handled by `TransfersScreen` and `TransferService.downloadTransfer()`.

Receiver sees incoming transfers

`TransferService.incomingTransfers(receiverId)` queries Firestore for transfers where:

- `receiverId == current user's uid`
- status is one of `uploading`, `uploaded`, `downloading`, `completed`, or `failed`

It returns a stream. The UI rebuilds when Firestore changes.

Accept or decline

If the transfer is `uploaded`, the receiver can accept or decline.

- Accept calls download logic.
- Decline updates the transfer status to `rejected`.

Download behavior

`downloadTransfer()`:

1. Stores previous status.
2. Marks transfer as `downloading`.
3. Creates local app directory.
4. Checks free storage through native `MethodChannel`.
5. Tries LAN download if sender LAN endpoint exists and appears reachable.
6. Falls back to Firebase Storage.
7. Writes files to disk.
8. Resolves filename conflicts.
9. Verifies SHA-256.
10. Updates Firestore progress.
11. Marks transfer as `completed`.

13. Where Files Are Stored

By default, files are saved under the app's documents directory:

`Documents/FluxDrop/{transferId}/filename`

This is app storage, meaning the app can access it reliably. The user can then open, share, save elsewhere, or save media to gallery/photos.

Why not always save directly to Downloads/Files?

Mobile storage is restricted. Android scoped storage and iOS sandboxing both limit arbitrary file writes.

The app uses a safer hybrid approach:

- Save inside FluxDrop app storage first.
- Let user open/share/save elsewhere.
- For images/videos, offer native save to Gallery/Photos.

14. Native MethodChannel Work

The channel name is:

`fluxdrop/storage`

Dart calls this channel from:

- `TransferService`
- `LanTransferService`
- `TransfersScreen`

Android native methods

Implemented in `MainActivity.kt`:

- `getWifiIp`: gets current Wi-Fi IP for LAN detection.

- `getFreeSpace`: checks available device storage.
- `saveToGallery`: saves images/videos using Android `MediaStore`.

iOS native methods

Implemented in `AppDelegate.swift`:

- `getFreeSpace`: checks available filesystem space.
- `saveToGallery`: saves images/videos using `PHPhotoLibrary`.

Why native code was used

The assessment rewarded platform-channel work. Also, some mobile behavior is naturally platform-specific:

- Android scoped storage needs `MediaStore`.
- iOS Photos saving needs `PHPhotoLibrary`.
- Wi-Fi IP detection is more reliable natively on Android.
- Free-space checks are platform-level operations.

15. Notification Flow

Notifications are handled by three parts:

- `NotificationService` in Flutter
- FCM token stored on user document
- `functions/index.js` Cloud Function

FCM token

When the app creates or restores a user, `AuthService` tries to get an FCM token on Android and stores it in Firestore under the user document.

The token tells Firebase where to send push notifications for that device.

Cloud Function trigger

`functions/index.js` watches Firestore:

`transfers/{transferId}`

When status changes to `uploaded`, it:

1. Reads the receiver user document.
2. Gets receiver FCM token.
3. Sends a push notification.
4. Includes transfer ID in notification data.

Android closed-app behavior

Android receives an explicit notification payload. This is important because data-only messages often do not display reliably when the app is killed.

When the user taps the notification, `NotificationService` emits a transfer-open request so the UI can open the Transfers tab.

iOS limitation

In this project, iOS FCM setup was intentionally limited in local/debug mode because APNs/signing behavior was unstable with a free Apple developer setup.

The honest explanation:

"Android is the primary closed-app notification demo path. iOS runs the app and transfer flow, but production-quality iOS push would require proper APNs capabilities and signing."

16. Firebase Components and Why They Are Used

Firestore

Used for anonymous identity.

Why:

- No email/password required.
- Gives a stable UID while local auth state exists.
- Fits the assessment requirement for anonymous onboarding.

Firestore

Used for:

- user documents
- short-code lookup
- transfer documents
- progress/status updates
- real-time listeners

Why:

- Built-in real-time streams.
- Good for small structured data.
- Easy to observe from both sender and receiver devices.

Firestore

Used for:

- uploaded file bytes
- receiver downloads

Why:

- Better for large files than Firestore.
- Works across distance and NAT.
- Avoids hosting a custom relay server.

Firestore

Used for:

- Android incoming transfer notification when app is closed/backgrounded.

Why:

- Push notifications are the correct way to alert a closed mobile app.

Cloud Functions

Used for:

- server-side notification trigger when transfer becomes `uploaded`.

Why:

- Sender should not directly send push to receiver.
- Backend can watch Firestore state and notify reliably.

17. Offline Behavior

Recipient offline behavior is queue-with-TTL.

If receiver is offline when sender uploads:

1. Transfer document remains in Firestore.
2. Files remain in Firebase Storage.
3. Transfer status stays `uploaded`.
4. Receiver can open app later and accept before expiry.
5. Expired transfer can be marked `expired`.

This is defensible for the assessment because the transfer does not require both phones to be online at the exact same moment.

18. Recovery After App Restart

`TransferService.recoverStaleTransfers(uid)` runs during startup.

It handles two common bad states:

Sender killed during upload

Firestore may be stuck at `uploading`.

Recovery marks it as `failed` with:

`Transfer interrupted - app was closed during upload. Please re-send.`

Receiver killed during download

Firestore may be stuck at `downloading`.

Recovery changes it back to `uploaded` so the receiver can try again.

This is not full background transfer survival. It is startup cleanup so the UI does not remain stuck forever.

19. LAN Fast-Path

The codebase includes an optional nearby transfer path in `LanTransferService`.

How it works:

1. Sender starts a local TCP server.
2. Sender writes `lanIp` and `lanPort` into the transfer document.

3. Receiver checks if it is on the same /24 subnet.
4. If yes, receiver tries direct TCP download.
5. If direct transfer fails, app falls back to Firebase Storage.

Why it exists:

- It shows a bonus nearby-transport attempt.
- It can be faster on same Wi-Fi.
- It mirrors the idea of local device-to-device transfer.

Important honesty:

The Firebase relay path is still the primary reliable path for demo and review.

20. UI Structure

HomeScreen

Main shell of the app.

It contains:

- user's short code card
- incoming ready card
- quick actions
- bottom navigation
- side drawer
- theme sheet
- connectivity chip

It also listens for incoming transfers so the home screen can show the most recent actionable incoming transfer.

SendScreen

Used to start a transfer.

Main responsibilities:

- recipient short-code entry
- invalid code handling
- file picking
- upload progress
- cancel upload
- success sheet

TransfersScreen

Used to manage transfer history.

Main responsibilities:

- incoming list
- outgoing list

- accept/decline
- download/cancel
- download again
- open file
- save/share elsewhere
- save media to gallery/photos

SettingsPanel

Used for app preferences.

Main responsibilities:

- saved path preference
- cache size and cleanup
- app/version/settings UI

AboutScreen

Used to explain project decisions and limitations.

This is useful in an interview because it mirrors the README in app form.

21. Theme System

The theme logic lives in `AppTheme`.

It supports:

- current/classic app theme
- NeoPOP visual mode using the official CRED neopop package through wrappers
- persistent theme choice via `SharedPreferences`
- system UI styling

Reusable UI wrappers live in `FluxButton` and `FluxSurface`.

Why wrappers were used:

- The app can switch visual styles without changing every screen manually.
- NeoPOP-specific components stay contained in shared widgets.

22. Important Edge Cases Handled

Case	How FluxDrop handles it
Invalid code	Firestore lookup returns null and UI shows clear error.
Short-code collision	Generation retries until a unique code is found.
Ambiguous characters	Alphabet avoids confusing characters.
Self-send	Send screen blocks sending to your own code.
Recipient offline	Transfer waits in Firestore with TTL.
Large files	500 MB limit.

Case	How FluxDrop handles it
Zero-byte files	Rejected before upload.
Multiple files	Stored as list of <code>FileInfo</code> ; aggregate progress tracked.
Network interruption	Firebase task behavior plus startup recovery.
Cancel upload	Active Firebase upload task can be cancelled.
Cancel download	Active Firebase download task can be cancelled safely.
Re-download cancel	Completed transfer returns to completed instead of pending.
Low storage	Native free-space check before download.
Filename conflict	App renames duplicates with suffix.
Corruption	SHA-256 mismatch rejects file.
App killed mid-transfer	Startup recovery prevents stuck UI states.
Closed app incoming notification	Android FCM + Cloud Function path.

23. Important Limitations

These are good to say honestly in the interview:

- The app is not strict MVVM.
- `TransferService` is doing too much and should be split for production.
- Full native background transfer continuation is not implemented.
- iOS closed-app push is not fully productionized under the local free Apple developer setup.
- LAN fast-path exists but Firebase relay is the main demo-safe path.
- Anonymous identity has no account recovery after app data clear.
- Extra at-rest encryption beyond Firebase/TLS is not implemented.
- No full production cleanup job for old Firebase Storage files.

24. Why Firebase Instead Of WebRTC

Firebase was chosen because the assessment required a working cross-distance transfer flow.

Firebase gives:

- auth
- real-time database updates
- file storage
- push notifications
- cloud triggers

WebRTC can be powerful, but reliable production WebRTC usually needs:

- signaling server
- STUN/TURN
- NAT handling
- more networking debugging

- fallback relay

For the assessment timeline, Firebase was the lower-risk choice.

25. If This Became Production, What Would Improve

Architecture improvements

- Split `TransferService` into smaller services:
- `TransferRepository`
- `UploadCoordinator`
- `DownloadCoordinator`
- `TransferRecoveryService`
- `StorageService`
- `NotificationRepository`
- Move screen orchestration into `ViewModels/Notifiers`.
- Use Riverpod, BLoC, or another team-standard state management pattern.
- Centralize Firestore writes for transfer status changes.

Reliability improvements

- Add Android foreground service for long uploads/downloads.
- Add iOS background `URLSession` for background transfers.
- Add server-side cleanup for expired transfers and Storage files.
- Add stronger retry/resume logic.
- Improve network transition handling.

Security improvements

- Add stronger abuse protection for short-code guessing.
- Add rate limits or blocklist.
- Add optional sender approval/contact trust model.
- Add encryption before uploading files if privacy requirements demand it.

iOS improvements

- Use paid Apple Developer account.
- Configure APNs entitlements properly.
- Test closed-app push on TestFlight.
- Harden iOS Photos/Files export behavior.

Testing improvements

- Unit tests for transfer state transitions.
- Integration tests for send/receive flow.
- Firebase emulator tests for Cloud Function behavior.
- Manual test matrix for Android/iOS, app foreground/background/closed.

26. How To Explain The Project In Interview

Short version:

"FluxDrop is a Flutter-based mobile file sharing app. Each user gets an anonymous short code. A sender enters the receiver's code, uploads files, and the receiver sees a real-time incoming transfer that can be accepted or declined. I used Firestore as the signaling and transfer-state layer, Firebase Storage for file relay, Cloud Functions plus FCM for Android closed-app notifications, and MethodChannels for storage checks and save-to-gallery/photos."

Architecture version:

"The project uses a pragmatic layered architecture: screens for UI, models for data, services for Firebase/native logic, and a small reusable widget layer. It is not strict MVVM. The main compromise is that `TransferService` became a large coordinator. If I extended the app, I would split that into smaller repositories/coordinators and move more UI orchestration into ViewModels or Notifiers."

Tradeoff version:

"I chose Firebase as the primary transport because it gave me a reliable cross-distance path without building my own relay backend. WebRTC would be interesting, but it would add signaling, TURN, NAT handling, and more failure modes. I focused on making the core mobile flow real and reviewable."

Session version:

"Firebase anonymous auth is the source of identity. If Firebase restores the same UID, FluxDrop loads the same Firestore user document and short code. If uninstall or platform behavior clears the UID, the app creates a new anonymous user and therefore a new short code. iOS may sometimes preserve auth/keychain state across local reinstall, while Android usually clears app data on uninstall."

27. Best Things To Defend Proudly

- You built an end-to-end mobile transfer flow.
- You handled anonymous identity and short-code lookup.
- You used Firestore listeners for real-time progress.
- You used Firebase Storage for actual files instead of abusing Firestore.
- You added native MethodChannel work.
- You handled cancellation and recovery cases.
- You documented limitations honestly.

28. Things To Never Overclaim

Do not say:

- "It is full MVVM."
- "Background transfer works perfectly."
- "iOS push is production-ready."
- "LAN transfer is the main reliable path."
- "Anonymous code always survives reinstall."

Better answers:

- "It is layered, not strict MVVM."
- "Startup recovery exists, but full background transfer survival is future work."

- "Android notification path is the main closed-app demo path."
- "Firebase relay is the primary reliable transport."
- "Identity persists while Firebase anonymous auth state is preserved."